

4.4 Complete Firmware Source Code Listing

The sketch below is exactly what was flashed onto both Node A and Node B — the hardware is symmetric, so the same code works on either side, and each node simply ends up transmitting and receiving on its own without needing to know which “side” it is.

```

#include <LiquidCrystal.h>
// LCD pin configuration (RS, E, D4, D5, D6, D7)
LiquidCrystal lcd(2, 3, 4, 5, 6, 7);

const int LASER_PIN = 12;
const int DETECTOR_PIN = 11;
const int BIT_PERIOD = 30; // Fixed 30 ms timing on both nodes
int charCount = 0; // Tracks how many characters have been printed on
the LCD

void setup() {
  Serial.begin(9600); // Baud rate locked at 9600
  pinMode(LASER_PIN, OUTPUT);
  pinMode(DETECTOR_PIN, INPUT);
  digitalWrite(LASER_PIN, LOW);

  // Initialise the LCD screen
  lcd.begin(16, 2);
  lcd.print("Li-Fi System");
  lcd.setCursor(0, 1);
  lcd.print("Ready...");

  Serial.println("\n=== SYSTEM RESET & READY ===");
  delay(1500);
  lcd.clear(); // Clear the welcome message and prepare the screen

  // Print a fixed heading on the first row so it need not be re-cleared
  lcd.setCursor(0, 0);
  lcd.print("Received Data:");
  lcd.setCursor(0, 1); // Move the cursor to the second row
}

void loop() {
  // --- Transmit section ---
  if (Serial.available() > 0) {
    char sendChar = Serial.read();
    sendByte(sendChar);
  }

  // --- Receive section ---
  if (digitalRead(DETECTOR_PIN) == HIGH) {
    char recvChar = receiveByte();

    // Filter out invalid characters and newline markers
    if (recvChar > 0 && recvChar != '\n' && recvChar != '\r') {
      Serial.print(recvChar); // Print to the Serial Monitor

      // Auto-clear the LCD's second row once 16 characters are filled
      if (charCount >= 16) {
        lcd.setCursor(0, 1);
        lcd.print(" "); // Erase the second row with spaces
        lcd.setCursor(0, 1); // Return the cursor to the row's
start
        charCount = 0;
      }

      // Print the received character sequentially on the LCD
      lcd.write(recvChar);
    }
  }
}

```

```

        charCount++; // Increment the character count
    }
}

// Modulates one 8-bit ASCII character into optical pulses
void sendByte(char c) {
    digitalWrite(LASER_PIN, HIGH); // Start Bit
    delay(BIT_PERIOD);

    for (int i = 0; i < 8; i++) {
        digitalWrite(LASER_PIN, bitRead(c, i));
        delay(BIT_PERIOD);
    }

    digitalWrite(LASER_PIN, LOW); // Stop Bit
    delay(BIT_PERIOD);
}

// Demodulates incoming optical pulses into one 8-bit ASCII character
char receiveByte() {
    char c = 0;
    delay(BIT_PERIOD + (BIT_PERIOD / 2)); // Delay to sample at the bit's centre

    for (int i = 0; i < 8; i++) {
        if (digitalRead(DETECTOR_PIN) == HIGH) {
            bitWrite(c, i, 1);
        } else {
            bitWrite(c, i, 0);
        }
        delay(BIT_PERIOD);
    }
    return c;
}

```